

Automated Daily Order Processing Pipeline

Architecture and Operations Manual

This manual describes the automated pipeline that ingests the daily order extract from cloud storage, validates its quality, transforms it for analytics, and alerts the operations team when anything goes wrong. It is written to be sufficient on its own: a reader who did not build the pipeline should be able to monitor it, interpret its alerts, and resolve its common failures using only this document.

Field	Value
Document title	Automated Daily Order Processing Pipeline — Architecture and Operations Manual
Version	1.0 (first release for operational handover)
Date issued	21 August 2026
Author	Data Engineering
Owner (accountable)	Data Engineering Lead
Operational owner	Operations Team — Logistics Analytics
Reviewers	Operations Manager; Platform / Cloud Team; Data Governance
Classification	Internal
Applies to environment	PROD (<code>adf-meridian-prod</code>). The same design is deployed to DEV and UAT with environment-specific resource names.
Review cycle	Every six months, or on any change to the pipeline, the quality rule set, or the alert routing.

Names of resources, mailboxes and teams in this document are the deployed PROD values. Where a value is expected to change between environments it is written as a parameter rather than a literal.

Contents

Contents.....	2
How to use this document.....	5
Access you need before you can operate the pipeline.....	5
Conventions used in this document	5
1. Pipeline Architecture	7
1.1 What the pipeline does.....	7
1.2 Components.....	7
1.3 How data moves	8
1.4 Activities and their dependencies.....	9
Parent pipeline — pl_Orchestrate_DailyOrders	10
Worker pipeline — pl_ProcessDailyOrders.....	10
1.5 Linked services and datasets.....	10
1.6 Pipeline parameters and variables.....	11
1.7 Design decisions.....	11
2. Scheduling and Trigger Configuration.....	13
2.1 Schedule at a glance	13
2.2 Trigger definition.....	13
2.3 Parameter mapping	14
2.4 Why the schedule is defined in UTC	15
2.5 Operating the trigger	15
2.6 Manual runs	15
3. Error Handling and Failure Notification	17
3.1 How failures are classified	17
3.2 Retry policy	17
3.3 Worked example — the retry behaviour of Copy_Landing_To_Raw	18
3.4 Where the failure handler lives, and why.....	18
3.5 The notification activity	19
3.6 What the operations team receives	20
3.7 The independent safety net.....	20
Metric alert — a run failed.....	21
Log alert — no run happened	21
3.8 What is deliberately not alerted	21
3.9 Guarantees that make a rerun safe	21
4. Data Quality Validation	23
4.1 How the assessment works	23

4.2 The rule set	23
4.3 The score and the threshold	23
4.4 The branching expression	24
4.5 What the notebook returns and where the detail goes	25
4.6 What happens to a rejected file.....	25
4.7 Changing the threshold.....	25
4.8 The validation notebook	26
5. Spark Transformation	28
5.1 The output contract	28
5.2 Notebook activity configuration	29
5.3 Cluster configuration	29
5.4 Parameters inside the notebook.....	30
5.5 What the transformation does	30
5.6 The transformation notebook.....	31
5.7 Reading the result back in the pipeline	31
5.8 Performance and layout	32
5.9 Schema changes.....	32
6. Operational Runbook.....	33
6.1 Daily health check	33
6.2 Where to look	33
6.3 Reading a failed run	33
6.4 Failure scenarios	34
S-01 The order file did not arrive.....	34
S-02 The file was rejected by the quality gate.....	35
S-03 The Databricks notebook failed.....	35
S-04 Storage or authentication failure	36
S-05 No run happened at all	36
6.5 Rerunning and backfilling.....	37
Rerunning a single failed run	37
Recovering a window with a fresh run	37
Backfilling several days	37
6.6 Escalation	38
6.7 Changes and maintenance.....	38
6.8 Things not to do	38
Appendix A — Diagnostic queries.....	40
Did the run happen, and how did it end?	40
Which activities failed, and with what error?.....	40

How is the quality score trending?40

Appendix B — Glossary.....41

Appendix C — References.....42

If page numbers appear as a placeholder, press Ctrl+A then F9 in Word to refresh the table of contents.

How to use this document

The pipeline is owned by Data Engineering and operated by the Operations Team. This document is the handover point between the two. It is organised so that each of the six sections answers one question an operator will eventually ask.

If you need to...	Read
Understand what the pipeline does and in what order	Section 1 — Pipeline Architecture
Know when it runs and what values it is given each day	Section 2 — Scheduling and Trigger Configuration
Understand what happens when something breaks, and who is told	Section 3 — Error Handling and Failure Notification
Explain why a file was rejected, or change the quality bar	Section 4 — Data Quality Validation
Understand what the Spark notebook does to the data	Section 5 — Spark Transformation
Respond to an alert at 03:00 and get the pipeline running again	Section 6 — Operational Runbook

Access you need before you can operate the pipeline

Request these through the standard access request process. Without them the runbook procedures in Section 6 cannot be completed.

Access	Role / permission	Used for
Azure Data Factory	Data Factory Contributor on <code>adf-meridian-prod</code>	Viewing runs in the Monitor tab, rerunning a failed run, stopping and starting the trigger
Data lake	Storage Blob Data Reader on <code>adlsmeridianprod</code>	Inspecting landing, raw and quarantine files
Azure Databricks	Workspace user with <code>can view</code> on <code>/Shared/orders</code>	Opening notebook run output and Spark logs
Log Analytics	Log Analytics Reader on <code>log-meridian-prod</code>	Running the diagnostic queries in Appendix A
Shared mailbox	Member of <code>ops-dataplatform@meridianlogistics.com</code>	Receiving pipeline alerts
Teams	Member of <code>Logistics Analytics – Pipeline Alerts</code>	Receiving alert cards and coordinating during an incident

Conventions used in this document

- **Fixed-width text** is an exact value: a resource name, an activity name, a folder path, or an expression that can be copied verbatim.
- Activity names match the names shown in the Data Factory Monitor tab exactly, so a name here can be searched for in the portal.
- Times are Coordinated Universal Time (UTC) unless a time zone is stated. The pipeline itself never uses local time.
- **Critical** and **Warning** are the two alert severities the pipeline emits; they are defined in Section 3.6.

Scope. This document covers the daily order pipeline only. The partner file transfer that places the file into the landing container, and the reporting layer that reads the curated table, are owned by other teams and are described here only where they touch the pipeline. Their contacts are in the escalation matrix in Section 6.6.

1. Pipeline Architecture

1.1 What the pipeline does

Once a day, a partner logistics system writes a single delimited order extract into the landing container of the data lake. The pipeline `pl_ProcessDailyOrders` collects that file at 02:00 UTC, keeps an immutable copy of it, scores it against a published set of data quality rules, and — only if it scores 80% or higher — transforms it through a PySpark notebook into the curated Delta table that all downstream reporting reads.

A file that scores below the threshold is not processed. It is moved to a quarantine folder alongside a report explaining which rules it failed, and the operations team is notified. No part of a rejected file reaches curated data. Separately, if any activity fails for a technical reason — the file never arrives, storage is unreachable, the Spark cluster cannot start — the pipeline stops and raises an alert within minutes.

The design follows five principles, which explain most of the decisions in this document:

- **Fail closed.** When the pipeline is unsure about the data, it stops and asks a human. It never publishes data it could not verify.
- **The raw zone is immutable.** Every file received is kept byte-for-byte under a dated path, so any day can be reprocessed without asking the partner to resend.
- **Reruns are safe.** Every step can be run again for the same business date without creating duplicates or double-counting.
- **Nothing fails silently.** Every terminal outcome — technical failure or quality rejection — produces an email and a row in the audit table.
- **Operators should not need the source code.** Every value an operator needs during an incident is emitted into the run output, the alert email, or the audit table.

1.2 Components

The pipeline is built from the following Azure resources. Every name below appears in the Azure portal exactly as written.

Component	Resource	Role in the pipeline
Orchestrator	<code>adf-meridian-prod</code> (Azure Data Factory)	Hosts the trigger, the pipelines and all activities. This is where an operator watches runs and reruns them.
Landing zone	<code>adlsmeridianprod</code> container <code>landing</code>	Where the partner drops the daily file. Written by the partner transfer job; read by the pipeline. The pipeline never deletes from here.
Raw zone	<code>adlsmeridianprod</code> container <code>raw</code>	Immutable dated copy of every file received. Retained 400 days. This is the source of truth for reprocessing.
Quarantine zone	<code>adlsmeridianprod</code> container <code>landing</code> , path <code>quarantine/</code>	Files that failed the quality gate, with their quality report. Retained 90 days.
Curated zone	<code>adlsmeridianprod</code> container <code>curated</code>	Delta tables that reporting reads, including <code>curated.fact_orders</code> .
Spark compute	<code>dbw-meridian-prod</code> (Azure Databricks)	Runs the quality-check notebook and the transformation notebook on short-lived job clusters.
Secret store	<code>kv-meridian-prod</code> (Azure Key Vault)	Holds the few secrets that are not covered by managed identity. No secret is stored in the pipeline definition.

Component	Resource	Role in the pipeline
Diagnostics	log-meridian-prod (Log Analytics)	Receives Data Factory run logs. Backs the queries in Appendix A and the missed-run alert.
Alerting	la-ops-alert-email (Logic App)	Turns an alert payload from the pipeline into an email and a Teams card.
Audit store	sqldb-meridian-meta table audit.pipeline_run	One row per pipeline run: outcome, quality score, row counts, duration.

1.3 How data moves

Figure 1 shows the components and the direction data flows between them. The upper row is the diagnostic and alerting path, which runs alongside the data path rather than inside it.

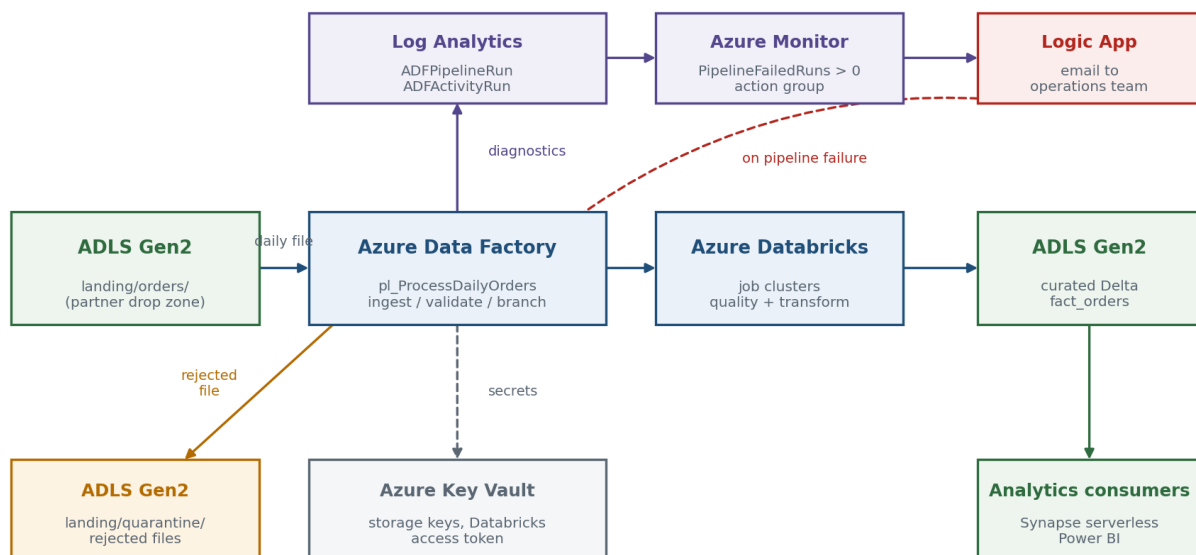


Figure 1 — Component architecture. Solid arrows carry data; dashed arrows carry telemetry and alerts.

The data lake is organised into four zones. Knowing which zone a file is in tells an operator how far the run got.

Zone	Path	Contents	Written by	Retention
Landing	landing/orders/yyyy/MM/	The partner's file exactly as delivered	Partner transfer job	30 days
Raw	raw/orders/ingest_date=yyyy-MM-dd/	Immutable copy of the file the pipeline actually processed	Activity 3, Copy_Landing_To_Raw	400 days
Quarantine	landing/quarantine/yyyy-MM-dd/	Rejected file plus quality_report.json	Activity 8, Quarantine_Rejected_File	90 days
Curated	curated/orders/fact_orders/	Delta table, partitioned by order_date	Activity 6, Transform_Orders	Indefinite

Reading the zones during an incident. If a file is in landing but not in raw, the run failed before or during the copy. If it is in raw but not in curated and not in quarantine, the run failed during quality checks or transformation. If it is in quarantine, the run worked correctly and the data was rejected — that is a data problem, not a pipeline problem.

1.4 Activities and their dependencies

The work is split across two pipelines. `pl_Orchestrate_DailyOrders` is a thin parent that does nothing except run the worker pipeline and catch its failure; `pl_ProcessDailyOrders` is the worker that does the actual processing. Section 3.4 explains why the failure handler lives in the parent rather than alongside the activities it protects.

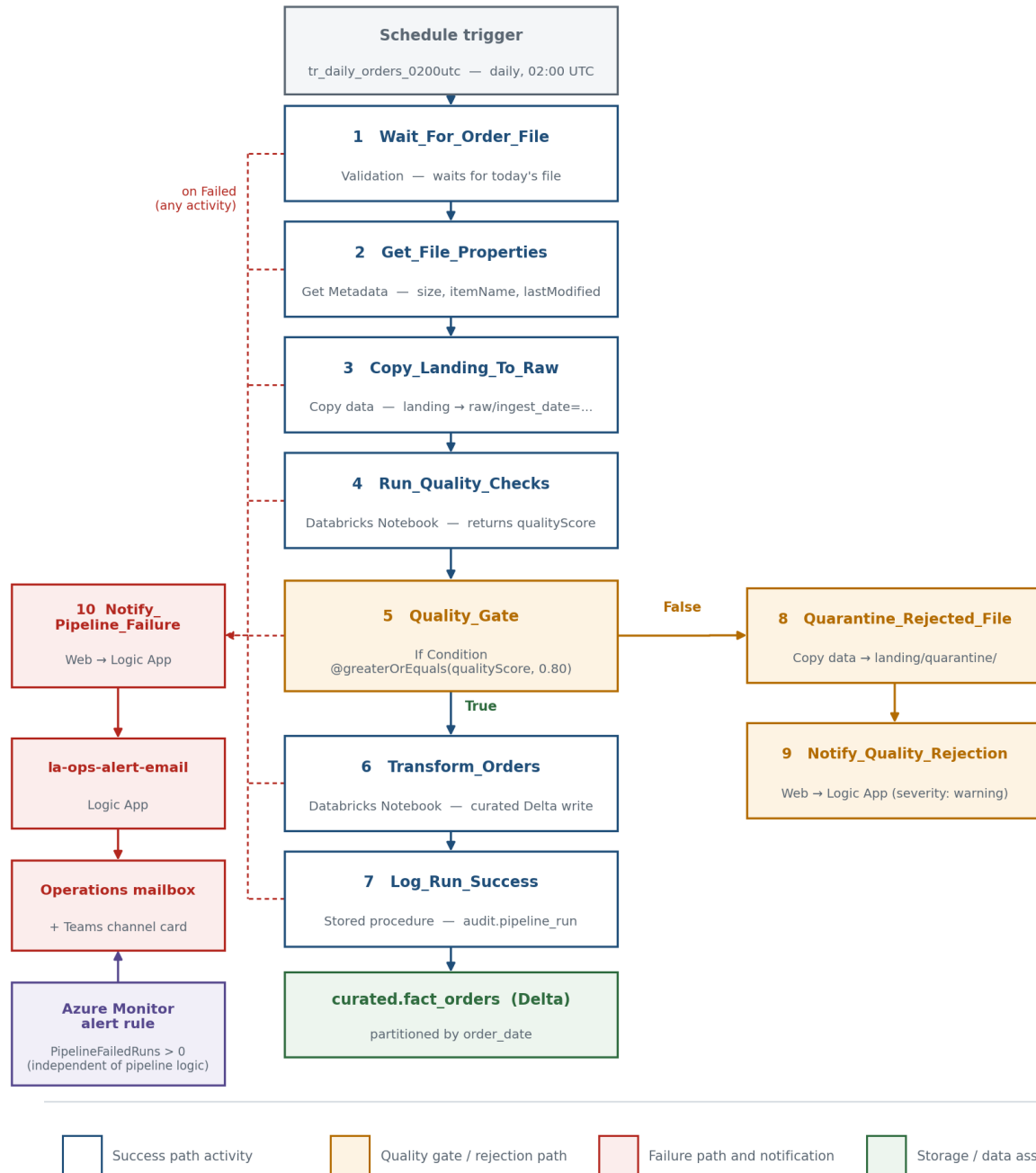


Figure 2 — Activity flow for `pl_ProcessDailyOrders`, with the quality branch and the failure path. Activity 10 belongs to the parent pipeline.

Parent pipeline — pl_Orchestrate_DailyOrders

#	Activity	Type	What it does	Depends on	Condition
P1	Run_Order_Processing	Execute Pipeline	Runs <code>pl_ProcessDailyOrders</code> and waits for it to finish, passing through every parameter received from the trigger.	—	—
P2	Notify_Pipeline_Failure	Web	Posts a Critical alert payload to the Logic App. Fires for any technical failure anywhere inside the worker pipeline.	P1	Failed

Worker pipeline — pl_ProcessDailyOrders

#	Activity	Type	What it does	Depends on	Condition
1	Wait_For_Order_File	Validation	Waits for the day's file to exist in landing and to be larger than zero bytes. Polls every 60 seconds and gives up after 90 minutes.	—	—
2	Get_File_Properties	Get Metadata	Reads <code>size</code> , <code>itemName</code> and <code>lastModified</code> so the alert email and audit row can report what was actually processed.	1	Succeeded
3	Copy_Landing_To_Raw	Copy data	Copies the file to the dated raw path without altering it. The landing copy is left in place for the partner's own reconciliation.	2	Succeeded
4	Run_Quality_Checks	Databricks Notebook	Runs the eight rules in Section 4 against the raw copy, writes the detailed report, and returns a quality score.	3	Succeeded
5	Quality_Gate	If Condition	Compares the returned score with the threshold and chooses the branch. Section 4.4 gives the expression.	4	Succeeded
6	Transform_Orders	Databricks Notebook	True branch. Cleanses, conforms and enriches the records and merges them into <code>curated.fact_orders</code> .	5 (true)	—
7	Log_Run_Success	Stored procedure	True branch. Writes the run outcome, score and row counts to <code>audit.pipeline_run</code> .	6	Succeeded
8	Quarantine_Rejected_File	Copy data	False branch. Copies the rejected file and its quality report to the dated quarantine path.	5 (false)	—
9	Notify_Quality_Rejection	Web	False branch. Posts a Warning alert payload naming the score and the rules that failed.	8	Completed

Activities 1 to 5 run strictly in sequence, each on the **Succeeded** condition of the one before, so the pipeline stops at the first problem rather than carrying a bad file forward. Activity 9 depends on activity 8 with the **Completed** condition rather than **Succeeded**, so the operations team is still told about a rejected file even if the quarantine copy itself fails.

1.5 Linked services and datasets

Linked services hold the connection details; datasets describe the shape and location of a file. Both are parameterised so a single dataset serves every day's file rather than one dataset per date.

Linked service	Type	Authentication	Points at
ls_adls_datalake	Azure Data Lake Storage Gen2	System-assigned managed identity of the data factory, granted Storage Blob Data Contributor	adlsmeridianprod

Linked service	Type	Authentication	Points at
ls_databricks_jobs	Azure Databricks	Managed identity; new job cluster per run	dbw-meridian-prod
ls_keyvault	Azure Key Vault	Managed identity, Get/List secret permission	kv-meridian-prod
ls_sql_metadata	Azure SQL Database	Managed identity	sqldb-meridian-meta

Dataset	Linked service	Parameters	Resolves to
ds_landing_orders_csv	ls_adls_datalake	folderPath, fileName	landing/orders/2026/08/orders_20260820.csv
ds_raw_orders_csv	ls_adls_datalake	ingestDate, fileName	raw/orders/ingest_date=2026-08-20/orders_20260820.csv
ds_quarantine_orders_csv	ls_adls_datalake	windowDate, fileName	landing/quarantine/2026-08-20/orders_20260820.csv

1.6 Pipeline parameters and variables

Parameters are supplied once when the run starts and never change during it; variables are set by activities as the run proceeds. Everything an operator might want to override for a single run is a parameter, so no rerun ever requires editing the pipeline.

Parameter	Type	Default	Meaning
windowDate	String	supplied by the trigger	The business date of the orders in the file, in yyyy-MM-dd. Drives every folder path and the curated partition. Not the same as the date the pipeline runs — see Section 2.3.
scheduledRunTime	String	supplied by the trigger	The scheduled fire time of the trigger in ISO 8601, e.g. 2026-08-21T02:00:00Z. Recorded in the audit row and shown in alert emails.
qualityThreshold	String	0.80	The minimum quality score a file must reach to be processed. Exposed as a parameter so a one-off override can be authorised without a code change (Section 4.7).
runMode	String	scheduled	One of <code>scheduled</code> , <code>manual</code> or <code>backfill</code> . Prefixes the alert subject and is recorded in the audit row so scheduled and operator-initiated runs can be told apart.

Variable	Type	Set by	Meaning
v_qualityScore	String	Activity 4	The score returned by the quality notebook, carried forward for logging and for the alert body.
v_rowsIngested	String	Activity 6	Rows merged into the curated table.
v_failedRules	String	Activity 4	Comma-separated rule IDs that did not reach full marks, e.g. DQ-05,DQ-07.

1.7 Design decisions

Where an obvious alternative exists, the reason for the choice is recorded here so that a future change is made deliberately.

Decision	Alternative considered	Why
Quality checks run in a Databricks notebook	Data Factory Data Flow expressions	The rules are expressed once, in code that can be unit-tested and reviewed, and the notebook can return a structured report. Data Flow would have scattered the rules across the pipeline definition.
Reject the whole file below 80%	Load the good rows and quarantine the bad ones	Orders arrive as a single daily statement and are reconciled as one. A partial load would make the curated table disagree with the partner's own totals, which is harder to detect than a missing day.
Immutable raw zone	Read directly from landing	Landing is owned by another team and is cleared after 30 days. Keeping our own copy means reprocessing never depends on the partner resending.
Failure handler in a parent pipeline	A failure path on each activity	Data Factory treats multiple dependencies on one activity as a logical AND, so a single handler wired to several activities would only fire if all of them failed. See Section 3.4.
Job clusters, not an all-purpose cluster	A long-running shared cluster	The pipeline runs for minutes a day. A job cluster costs nothing between runs and isolates this workload from interactive users.
MERGE into curated, not APPEND	Append and de-duplicate later	Makes a rerun of the same business date a no-op rather than a duplication incident. This is what makes the rerun procedure in Section 6.5 safe.

2. Scheduling and Trigger Configuration

The pipeline is started by one schedule trigger and nothing else. No person starts it in normal operation, and no other pipeline calls it. This section documents when it fires, what values it hands to the pipeline, and how an operator stops and starts it safely.

2.1 Schedule at a glance

Property	Value
Trigger name	tr_daily_orders_0200utc
Trigger type	Schedule trigger
Recurrence	Every 1 day at 02:00:00
Time zone	UTC — set explicitly, never the factory's regional default
Pipeline started	p1_Orchestrate_DailyOrders (the parent)
First scheduled run	2026-09-01T02:00:00Z
End time	None. The trigger runs until it is explicitly stopped.
Concurrency	Pipeline concurrency is set to 1. If a run is still going when the next fires, the new run queues rather than running alongside it.
Typical duration	12–20 minutes for a file of roughly 120,000 rows
Data freshness commitment	curated.fact_orders is complete for the previous business day by 03:00 UTC
Latest acceptable file arrival	03:30 UTC. Activity 1 waits 90 minutes from 02:00 and then fails the run.

2.2 Trigger definition

This is the deployed definition. It is source-controlled in `adf-meridian/triggers/tr_daily_orders_0200utc.json` and released through the standard deployment pipeline; it is not edited by hand in the portal.

```
{
  "name": "tr_daily_orders_0200utc",
  "properties": {
    "type": "ScheduleTrigger",
    "runtimeState": "Started",
    "typeProperties": {
      "recurrence": {
        "frequency": "Day",
        "interval": 1,
        "startTime": "2026-09-01T02:00:00Z",
        "timeZone": "UTC",
        "schedule": {
          "hours": [ 2 ],
          "minutes": [ 0 ]
        }
      }
    }
  },
  "pipelines": [
    {
      "pipelineReference": {
        "type": "PipelineReference",
        "referenceName": "pl_Orchestrate_DailyOrders"
      },
      "parameters": {
        "windowDate": "@formatDateTime(addDays(trigger().scheduledTime, -1), 'yyyy-MM-dd')",
        "scheduledRunTime": "@trigger().scheduledTime",
        "qualityThreshold": "0.80",
        "runMode": "scheduled"
      }
    }
  ]
}
```

Two details that are easy to get wrong. The `startTime` must carry the trailing `Z` when the time zone is UTC; without it the value is read as a local time. And `timeZone` must be stated explicitly — leaving it out makes the schedule follow a regional default that can shift with daylight saving.

2.3 Parameter mapping

Every value the pipeline needs is supplied by the trigger. Nothing is read from the system clock inside the pipeline, which is what makes a rerun for an earlier date behave exactly like the original run.

Trigger expression	Pipeline parameter	Example value	Consumed by
<code>@formatDateTime(addDays(trigger().scheduledTime, -1), 'yyyy-MM-dd')</code>	<code>windowDate</code>	2026-08-20	Source and raw folder paths, quarantine path, notebook <code>baseParameters</code> , curated partition, audit row
<code>@trigger().scheduledTime</code>	<code>scheduledRunTime</code>	2026-08-21T02:00:00Z	Audit row, alert email subject and body
literal 0.80	<code>qualityThreshold</code>	0.80	The <code>Quality_Gate</code> If Condition expression (Section 4.4)
literal <code>scheduled</code>	<code>runMode</code>	<code>scheduled</code>	Alert subject prefix, audit row, notebook logging

The run date and the business date are not the same. The run that fires at 02:00 UTC on 21 August processes the orders for **20 August**. That is why `windowDate` is derived with `addDays(..., -1)`. When an operator is asked to reprocess "Thursday's orders", `windowDate` is Thursday's date, not the date of the run that failed. Getting this wrong reprocesses the wrong day, which the pipeline cannot detect for you.

`windowDate` reaches the source file path through the dataset parameters on `ds_landing_orders_csv`:

```

folderPath:
  @concat('orders/',
    formatDateTime(pipeline().parameters.windowDate, 'yyyy/MM'))

fileName:
  @concat('orders_',
    formatDateTime(pipeline().parameters.windowDate, 'yyyyMMdd'),
    '.csv')

resolved for windowDate = 2026-08-20:
  landing/orders/2026/08/orders_20260820.csv

```

2.4 Why the schedule is defined in UTC

The trigger, the file name, the folder paths and the curated partition all use UTC. The partner produces the extract on a UTC schedule, and the reporting layer reports on UTC business dates, so there is no point in the chain where a local time would be correct.

This also removes daylight saving as a failure mode. A trigger defined in a zone that observes daylight saving either skips a run or fires twice on changeover days; a UTC trigger fires exactly once every 24 hours, all year. If a future request asks for the pipeline to run at a local business time, the change should be made by adjusting the UTC hour rather than by changing the trigger's time zone.

2.5 Operating the trigger

A trigger must be stopped before its definition can be changed, and started again afterwards. Publishing a change to a running trigger is rejected.

Action	Command	When it is used
Stop	<code>az datafactory trigger stop --resource-group rg-meridian-prod --factory-name adf-meridian-prod --name tr_daily_orders_0200utc</code>	Before a planned maintenance window, or when the source system is known to be down and repeated failure alerts would be noise
Start	<code>az datafactory trigger start --resource-group rg-meridian-prod --factory-name adf-meridian-prod --name tr_daily_orders_0200utc</code>	After maintenance. Always confirm the state afterwards.
Confirm state	<code>az datafactory trigger show --resource-group rg-meridian-prod --factory-name adf-meridian-prod --name tr_daily_orders_0200utc --query properties.runtimeState</code>	After any stop or start, and as part of the daily health check

A stopped trigger does not catch up. Schedule triggers only fire forward. If the trigger is stopped over a window — or the factory is unavailable — that day's run is simply never created, and no alert is raised by the pipeline because the pipeline never started. This is exactly the gap the missed-run alert in Section 3.7 exists to close, and any window missed this way must be recovered with the backfill procedure in Section 6.5.

2.6 Manual runs

An operator starts a run by hand in two situations: to recover a missed or failed window, and to reprocess a day after the partner has corrected a file. Both use the same mechanism — trigger `p1_Orchestrate_DailyOrders` manually and supply the parameters explicitly.

1. In the Data Factory studio, open **Author**, select `p1_Orchestrate_DailyOrders`, and choose **Debug** for a test or **Add trigger → Trigger now** for a real run.
2. Set `windowDate` to the **business date** being recovered, not today's date.
3. Set `runMode` to `manual` for a single recovery, or `backfill` when replaying a range of dates. This is what keeps recovery runs distinguishable from scheduled ones in the audit table.
4. Leave `qualityThreshold` at `0.80` unless an override has been authorised under Section 4.7.

5. Set `scheduledRunTime` to the timestamp of the window being recovered, e.g. `2026-08-21T02:00:00Z`, so the audit row lines up with the window it replaces.

Because the raw copy overwrites the same dated path and the curated write is a merge on `order_id`, running the same `windowDate` more than once produces the same result as running it once. Section 3.9 sets out the guarantees this relies on.

3. Error Handling and Failure Notification

The pipeline distinguishes between problems it should retry, problems it should stop for, and problems that are not pipeline failures at all. This section sets out that distinction, the retry policy applied to each activity, and the two independent routes by which a failure reaches a human.

3.1 How failures are classified

Class	Typical examples	Handled by	What the operator does
Transient infrastructure	Storage throttling (HTTP 503), a dropped connection, a slow Databricks cluster acquisition	The activity retry policy. Most are invisible because a retry succeeds.	Nothing. A run that succeeded after retries is a healthy run.
Terminal technical	Expired credential, missing notebook, malformed path, cluster quota exhausted, node type unavailable	Retries are exhausted, the activity fails, the pipeline fails, a Critical alert is sent.	Follow the matching scenario in Section 6.4.
Source availability	The partner file has not arrived by 03:30 UTC	Activity 1 times out and fails; Critical alert.	Scenario S-01.
Data quality	The file arrived and is readable, but scores below the threshold	The quality gate takes the false branch. This is not a pipeline failure; a Warning alert is sent.	Scenario S-02. The pipeline is working as designed.

A rejected file is a successful run. When a file fails the quality gate, the pipeline finishes with status Succeeded, because it did exactly what it was built to do. Operators should not look for a failed run in the Monitor tab after receiving a quality Warning — there will not be one.

3.2 Retry policy

Every activity carries an explicit policy rather than the Data Factory defaults. Retries are set where a second attempt has a realistic chance of succeeding, and left at zero where it does not — repeatedly retrying a genuinely broken activity only delays the alert.

Activity	timeout	retry	interval (s)	Why this setting
Wait_For_Order_File	01:30:00	0	—	The activity is itself a wait. Retrying it would restart the 90-minute wait and delay the alert by hours.
Get_File_Properties	00:10:00	2	30	A cheap metadata read; a short retry absorbs momentary storage unavailability.
Copy_Landing_To_Raw	01:00:00	3	120	The most common place for transient failures. Three attempts two minutes apart cover a typical storage throttling episode.
Run_Quality_Checks	01:00:00	1	180	One retry covers a failed cluster acquisition. If the notebook itself is broken, a second retry only wastes cluster minutes.
Quality_Gate	00:10:00	0	—	Control flow only. It evaluates an expression; there is nothing to retry.
Transform_Orders	02:00:00	1	300	The merge is idempotent, so a retry is safe. Five minutes gives a replacement cluster time to come up.
Log_Run_Success	00:10:00	2	30	A short database call; retries cover a brief connection failure so a healthy run is not marked failed by its own logging step.
Quarantine_Rejected_File	00:30:00	2	60	Same storage risks as the raw copy, on a smaller file.
Notify_Quality_Rejection Notify_Pipeline_Failure	00:05:00	2	15	The alert must survive a transient Logic App throttle. <code>secureOutput</code> is set to <code>true</code> so the payload is not written to run logs.

3.3 Worked example — the retry behaviour of Copy_Landing_To_Raw

This activity is documented in full because it is the one an operator will most often see retrying, and because its behaviour is representative of the others.

```
{
  "name": "Copy_Landing_To_Raw",
  "type": "Copy",
  "dependsOn": [
    { "activity": "Get_File_Properties",
      "dependencyConditions": [ "Succeeded" ] }
  ],
  "policy": {
    "timeout": "01:00:00",
    "retry": 3,
    "retryIntervalInSeconds": 120,
    "secureInput": false,
    "secureOutput": false
  },
  "typeProperties": {
    "source": { "type": "DelimitedTextSource" },
    "sink": { "type": "DelimitedTextSink" },
    "validateDataConsistency": true,
    "enableSkipIncompatibleRow": false
  },
  "inputs": [ { "referenceName": "ds_landing_orders_csv", "type": "DatasetReference" } ],
  "outputs": [ { "referenceName": "ds_raw_orders_csv", "type": "DatasetReference" } ]
}
```

retry: 3 means one initial attempt followed by up to three more — four attempts in total. A run in which storage is throttling looks like this in the Monitor tab:

Attempt	Clock (UTC)	Outcome
1	02:04:10	Fails with <code>ErrorCode=AdlsGen2OperationFailed</code> , HTTP 503 (server busy)
2	02:06:10	Fails the same way. The 120-second interval is measured from the end of the previous attempt.
3	02:08:10	Succeeds. The activity reports Succeeded , and the run continues normally.
4	—	Not reached. Had attempt 3 also failed, this would be the final attempt; if it failed too, the activity would be marked Failed.

If all four attempts fail, `Copy_Landing_To_Raw` reports Failed. Activity 4 depends on it with the `Succeeded` condition, so activity 4 and everything after it is skipped, the worker pipeline reports Failed, and the parent pipeline's failure path fires. On the timings above the operations mailbox receives the alert at roughly 02:12 — about twelve minutes after the trigger fired.

Reading run durations. The timeout bounds a single attempt, and the retry policy then governs how many attempts are made. When judging whether an activity is stuck, allow for the worst case of four attempts plus the intervals between them, not for a single timeout period.

3.4 Where the failure handler lives, and why

Data Factory evaluates multiple dependencies on a single activity as a logical **AND**. A single notification activity wired to the failure output of six activities would therefore only run if all six had failed — which never happens, because the chain stops at the first one. The consequence is that the obvious design silently never alerts.

Two designs avoid this. The first is a separate notification activity on each activity's failure output, which works but duplicates the same configuration six times and is easy to leave out when a new activity is added. The second, used here, is to put the processing in a worker pipeline and call it from a parent that has exactly one failure path. Any failure anywhere in the worker propagates to the Execute Pipeline activity, and one handler covers all of it — including activities added in future.

```
{
  "name": "pl_Orchestrate_DailyOrders",
  "properties": {
    "activities": [
      {
        "name": "Run_Order_Processing",
        "type": "ExecutePipeline",
        "policy": { "secureInput": false },
        "typeProperties": {
          "pipeline": {
            "referenceName": "pl_ProcessDailyOrders",
            "type": "PipelineReference"
          },
          "waitOnCompletion": true,
          "parameters": {
            "windowDate": "@pipeline().parameters.windowDate",
            "scheduledRunTime": "@pipeline().parameters.scheduledRunTime",
            "qualityThreshold": "@pipeline().parameters.qualityThreshold",
            "runMode": "@pipeline().parameters.runMode"
          }
        }
      },
      {
        "name": "Notify_Pipeline_Failure",
        "type": "WebActivity",
        "dependsOn": [
          { "activity": "Run_Order_Processing",
            "dependencyConditions": [ "Failed" ] }
        ]
      }
    ]
  }
}
```

`waitOnCompletion` must stay `true`. If it is set to false the Execute Pipeline activity reports success as soon as the worker starts, the parent finishes immediately, and no failure is ever detected. This single property is what makes the whole failure path work.

3.5 The notification activity

`Notify_Pipeline_Failure` posts a JSON payload to the Logic App. Every field it sends is chosen so that the resulting email answers an operator's first three questions — what failed, for which business date, and where do I look — without anyone having to open the portal first.

```
{
  "name": "Notify_Pipeline_Failure",
  "type": "WebActivity",
  "dependsOn": [
    { "activity": "Run_Order_Processing", "dependencyConditions": [ "Failed" ] }
  ],
  "policy": {
    "timeout": "00:05:00",
    "retry": 2,
    "retryIntervalInSeconds": 15,
    "secureOutput": true
  },
  "typeProperties": {
    "method": "POST",
    "url": "@concat(pipeline().globalParameters.alertLogicAppUrl)",
    "headers": { "Content-Type": "application/json" },
    "body": {
      "severity": "Critical",
      "environment": "PROD",
      "runMode": "@{pipeline().parameters.runMode}",
      "factoryName": "@{pipeline().DataFactory}",
      "pipelineName": "@{pipeline().Pipeline}",
      "parentRunId": "@{pipeline().RunId}",
      "childRunId": "@{activity('Run_Order_Processing').output.pipelineRunId}",
      "windowDate": "@{pipeline().parameters.windowDate}",
      "scheduledTime": "@{pipeline().parameters.scheduledRunTime}",
      "errorMessage": "@{activity('Run_Order_Processing').Error.message}",
      "monitorUrl": "@concat('https://adf.azure.com/en/monitoring/pipelineRuns/',
        activity('Run_Order_Processing').output.pipelineRunId,
        '?factory=', pipeline().DataFactory)"
    }
  }
}
```

Payload field	Where the value comes from	Why the operator needs it
errorMessage	The error returned by the failed activity inside the worker pipeline	Names the failing activity and the underlying error code — usually enough to pick the right runbook scenario without opening the portal
childRunId	<code>activity('Run_Order_Processing').output.pipelineRunId</code>	The run ID of the worker pipeline. This is the ID to search for in the Monitor tab; the parent's run ID will not find the failing activity.
windowDate	Pipeline parameter	The business date to use when rerunning. Section 2.3 explains why this is not the date of the run.
monitorUrl	Composed from the child run ID and factory name	A direct link to the failed run, so the email is one click from the diagnosis
runMode	Pipeline parameter	Tells the reader whether this was the nightly schedule or somebody's recovery attempt

3.6 What the operations team receives

The Logic App `la-ops-alert-email` accepts the payload on an HTTP request trigger, formats it, and delivers it to the operations mailbox and the Teams channel. It sends one message per payload; it does not aggregate or suppress, because the pipeline only sends one payload per run.

Severity	Sent when	Goes to	Expected response
Critical	Any technical failure of the pipeline, including the file failing to arrive	<code>ops-dataplatform@meridianlogistics.com</code> , the Teams alert channel, and the on-call rotation between 22:00 and 06:00 UTC	Acknowledge within 30 minutes; curated data is missing for the day until it is resolved
Warning	A file was received and processed but failed the quality gate	Operations mailbox and the Teams alert channel only. No on-call page.	Review during the working day; contact the partner if the pattern repeats
Info	A manual or backfill run completed successfully	Teams alert channel only	None. Recorded so that recovery work is visible to the next shift.

A Critical email renders as follows. The subject line alone is designed to be actionable on a phone.

```
Subject: [CRITICAL] PROD order pipeline failed - business date 2026-08-20

Pipeline      pl_ProcessDailyOrders (scheduled run)
Business date 2026-08-20
Scheduled     2026-08-21T02:00:00Z
Failed at     2026-08-21T02:12:41Z

Error
  Activity 'Copy_Landing_To_Raw' failed:
  ErrorCode=AdlsGen2OperationFailed, Operation returned an invalid
  status code 'ServiceUnavailable' after 4 attempts.

Run           b41f8e02-77a1-4c6e-9a53-2f6d0c9e1a77
Open the run  https://adf.azure.com/en/monitoring/pipelineruns/b41f8e02-...

Curated data for 2026-08-20 is not available until this run completes.
Runbook: Architecture and Operations Manual, Section 6.4.
```

3.7 The independent safety net

The Web activity described above cannot fire in the one case that matters most: when the pipeline never ran at all. If the trigger is stopped, the factory is unavailable, or the run is never created, there is no activity to execute and no alert to send. Two Azure Monitor rules close that gap, and they run entirely outside the pipeline.

Metric alert — a run failed

Setting	Value
Rule name	alert-adf-pipeline-failed-prod
Scope	adf-meridian-prod
Signal	PipelineFailedRuns (Failed pipeline runs metrics)
Condition	Total is greater than 0
Aggregation granularity	15 minutes, evaluated every 5 minutes
Severity	Sev 1
Action group	ag-ops-oncall — email to the operations mailbox, SMS to the on-call phone, Teams webhook
Purpose	Catches failures the in-pipeline alert could not report, including a failure of the notification activity itself

Log alert — no run happened

This is the rule that detects silence. It runs once at 03:15 UTC and fires if no run of the worker pipeline has been recorded since 01:45 UTC.

```
ADFPipelineRun
| where TimeGenerated between (ago(90m) .. now())
| where PipelineName == "pl_ProcessDailyOrders"
| where Status in ("Succeeded", "Failed", "InProgress", "Queued")
| summarize runs = count()
| where runs == 0
```

The rule is configured with **Number of results greater than 0** so that the absence of a run produces a row, and therefore an alert, routed to [ag-ops-oncall](#) at Sev 1. Without it, a stopped trigger would be discovered only when somebody noticed missing data in a report.

3.8 What is deliberately not alerted

- **Activity retries that eventually succeed.** These are the retry policy doing its job. They are visible in the run detail and in Log Analytics, but they do not raise anything.
- **Individual quality rules failing.** A file can fail several rules and still score above 80%. The failed rule IDs are recorded in the quality report and the audit row; only the overall verdict is alerted on.
- **Runs where `runMode` is `backfill`.** Backfills are run deliberately by an operator who is already watching. They still write audit rows and quality reports, and a failure is still reported in the run history, but they do not page the on-call rotation.

This restraint is intentional. An alert channel that carries routine noise stops being read, and the missed-run alert in particular depends on being trusted.

3.9 Guarantees that make a rerun safe

Recovery procedures in Section 6 assume it is always safe to run a business date again. That assumption rests on four properties of the design:

- The raw copy writes to a path derived from `windowDate`, so a rerun **overwrites** the same file rather than adding another.
- The curated write is a `MERGE` keyed on `order_id`, so re-processing the same records updates them in place instead of duplicating them.

- The quality report is written to a path derived from `windowDate` and is likewise overwritten, so the latest report always describes the latest attempt.
- The audit row is upserted on `(window_date, run_mode)`, so the audit table records the outcome of the most recent attempt for a date while still preserving the distinction between the scheduled run and any recovery run.

The one thing that is **not** safe is running two different business dates concurrently against the same curated partition. Pipeline concurrency is set to 1 to prevent this, and backfills are run sequentially for the same reason (Section 6.5).

4. Data Quality Validation

Quality validation is the gate that decides whether a day's orders are allowed into curated data. It runs after the file has been safely copied to the raw zone and before any transformation, so the bytes that were assessed are exactly the bytes that were kept, and nothing has yet been written anywhere a report could read it.

4.1 How the assessment works

Activity 4, [Run_Quality_Checks](#), executes the notebook [/Shared/orders/01_quality_checks](#) against the raw copy. The notebook evaluates eight rules, computes a single weighted score between 0 and 1, writes a detailed report to the curated zone, and returns a small summary to the pipeline. The pipeline compares the score to the threshold and branches. The notebook itself never decides the outcome — it only measures — which keeps the decision visible in the pipeline where an operator can see it.

4.2 The rule set

Rules are versioned with the notebook and are listed here as deployed. Weights reflect how much damage a failure of that rule would do downstream, not how often it fails.

ID	Rule	Dimension	Scope	Weight	Effect of failure
DQ-01	The header row matches the agreed 14 columns, in the agreed order and spelling.	Validity	File	25%	Scores 0 or 1 for the whole file — there is no partial credit for a schema.
DQ-02	<code>order_id</code> is present and unique within the file.	Uniqueness	Row	20%	Duplicate and null IDs are counted as failing rows.
DQ-03	The mandatory fields <code>order_id</code> , <code>customer_id</code> , <code>order_date</code> , <code>ship_to_country</code> , <code>quantity</code> and <code>unit_price</code> are all non-null.	Completeness	Row	20%	A row failing any mandatory field counts once against this rule.
DQ-04	<code>order_date</code> parses as <code>yyyy-MM-dd</code> , is not in the future, and is not more than 30 days old.	Timeliness	Row	10%	Catches both malformed dates and a partner replaying an old extract.
DQ-05	<code>quantity</code> is a whole number greater than zero.	Validity	Row	10%	Zero and negative quantities are the most common source of wrong revenue figures.
DQ-06	<code>unit_price</code> is numeric, at least 0 and less than 100,000.	Validity	Row	5%	The upper bound catches decimal-separator and currency-unit errors.
DQ-07	<code>ship_to_country</code> is a valid ISO 3166-1 alpha-2 code present in the reference list.	Consistency	Row	5%	Prevents unroutable records reaching the shipping dashboards.
DQ-08	<code>status</code> is one of <code>CREATED</code> , <code>PICKED</code> , <code>SHIPPED</code> , <code>DELIVERED</code> , <code>CANCELLED</code> .	Consistency	Row	5%	An unrecognised status silently breaks the fulfilment funnel report.

Two conditions bypass scoring entirely and force an immediate rejection with a score of 0: a file that cannot be parsed at all, and a file containing zero data rows. Both are reported as `verdict: REJECT` with the reason stated in the quality report.

4.3 The score and the threshold

Each rule contributes its weight multiplied by the proportion of rows that passed it:

```
score = SUM over rules of ( weight_r x pass_rate_r )

where pass_rate_r = passing_rows / total_rows      (row-level rules)
      pass_rate_r = 1 if the rule holds, else 0    (file-level rules)

weights sum to 1.00, so score is always between 0.00 and 1.00
```

The threshold is **0.80**. A file scoring 0.80 or above is transformed; a file below it is quarantined. The comparison is inclusive, so a file scoring exactly 0.80 is accepted.

The table below contrasts a healthy day with a day that is rejected, using real proportions from the two cases.

Rule	Weight	20 Aug — pass rate	Contribution	22 Aug — pass rate	Contribution
DQ-01	0.25	1.0000	0.2500	1.0000	0.2500
DQ-02	0.20	0.9990	0.1998	0.8000	0.1600
DQ-03	0.20	0.9850	0.1970	0.4000	0.0800
DQ-04	0.10	0.9950	0.0995	0.8500	0.0850
DQ-05	0.10	0.9800	0.0980	0.6000	0.0600
DQ-06	0.05	0.9992	0.0500	0.9500	0.0475
DQ-07	0.05	0.9700	0.0485	0.5000	0.0250
DQ-08	0.05	0.9917	0.0496	0.9000	0.0450
Score	1.00		0.9924 — pass		0.7525 — reject

A schema change always rejects the file. DQ-01 carries 25% of the total weight, so a file whose header does not match the agreed contract can reach at most 0.75 and is rejected no matter how clean its data is. This is deliberate: an unexpected column is a change to the interface between two organisations, and it should be agreed rather than absorbed silently. Scenario S-02 in the runbook covers what to do when this happens.

4.4 The branching expression

Activity 5 is an If Condition. Its expression is the single line that decides whether a day's data is published.

```
{
  "name": "Quality_Gate",
  "type": "IfCondition",
  "dependsOn": [
    { "activity": "Run_Quality_Checks", "dependencyConditions": [ "Succeeded" ] }
  ],
  "typeProperties": {
    "expression": {
      "type": "Expression",
      "value": "@greaterOrEquals(
        float(activity('Run_Quality_Checks').output.runOutput.qualityScore),
        float(pipeline().parameters.qualityThreshold)
      )"
    },
    "ifTrueActivities": [ /* Transform_Orders, Log_Run_Success      */ ],
    "ifFalseActivities": [ /* Quarantine_Rejected_File, Notify_... */ ]
  }
}
```

Part of the expression	What it does
<code>activity('Run_Quality_Checks').output.runOutput</code>	The JSON object the notebook returned through <code>dbutils.notebook.exit</code> . This is the only channel by which a Databricks notebook can hand a value back to the pipeline.
<code>.qualityScore</code>	The single field the gate depends on. It is a string in the returned JSON, because <code>runOutput</code> values always arrive as strings.

Part of the expression	What it does
<code>float(...)</code>	Converts both sides to numbers. Without it the comparison is made on text, where "0.9" sorts after "0.80" but "0.1" sorts before it — a subtle and dangerous bug.
<code>pipeline().parameters.qualityThreshold</code>	The threshold is a parameter, not a literal, so a one-off authorised override changes the run rather than the pipeline definition.
<code>greaterOrEquals</code>	Inclusive. A score of exactly 0.80 passes. Use <code>greater</code> instead if the intent ever changes to strictly above.

4.5 What the notebook returns and where the detail goes

The value returned to the pipeline is deliberately small — a summary an operator can read in an alert email. Row-level detail is far too large to return and is written to the curated zone instead.

```
{
  "verdict": "REJECT",
  "qualityScore": "0.7525",
  "threshold": "0.80",
  "rowsRead": "118420",
  "rowsPassingAllRules": "41230",
  "failedRules": "DQ-02,DQ-03,DQ-05,DQ-07",
  "worstRule": "DQ-03",
  "schemaHash": "8f3c9d21b7e4a05c",
  "reportPath": "abfss://curated@adlsmeridianprod.dfs.core.windows.net/
    quality/orders/window_date=2026-08-22/quality_report.json"
}
```

Keep the return value small. Databricks caps the value returned through `dbutils.notebook.exit` at 2 MB. Returning per-row failures would exceed that on a bad day — exactly the day the pipeline most needs to work. The summary above is a few hundred bytes regardless of file size.

The full report at `reportPath` contains, for each rule: the rule ID and description, rows evaluated, rows passed, the pass rate, the weighted contribution, and up to 100 example failing `order_id` values. Those examples are what the operations team sends to the partner when a file is rejected.

4.6 What happens to a rejected file

1. The gate takes the false branch. Nothing is written to `curated.fact_orders`; the previous day's data remains the most recent in the table.
2. `Quarantine_Rejected_File` copies the file and its quality report to `landing/quarantine/<windowDate>.`
3. `Notify_Quality_Rejection` sends a **Warning** alert naming the score, the threshold, and the failed rule IDs.
4. The worker pipeline finishes with status **Succeeded** — see the note in Section 3.1.
5. The operations team follows scenario S-02: read the report, identify the pattern, and raise it with the partner.
6. When a corrected file is delivered, it is placed in landing under the original name and the day is reprocessed with `runMode` set to `manual` (Section 6.5).

4.7 Changing the threshold

The 0.80 threshold is a governance decision, not a technical one. It is owned by the Data Governance forum and may only be changed with its agreement.

Change	How	Who approves
Permanent change to the standard	Change the default value of <code>qualityThreshold</code> in the pipeline definition and release it through the normal deployment pipeline	Data Governance forum, recorded in the forum minutes
One-off override for a single run	Supply a different <code>qualityThreshold</code> on a manual run	Operations Manager and Data Engineering Lead, both recorded in the incident ticket

Change	How	Who approves
Changing a rule or its weight	Change the notebook and its unit tests; weights must still sum to 1.00	Data Governance forum; Data Engineering implements

An override is visible afterwards. The threshold actually applied is written to the audit row for every run. A run that published data under a relaxed threshold can therefore be identified later, which is the point — the record should outlive the conversation that authorised it.

4.8 The validation notebook

The rules are expressed as a table of named predicates, so adding a rule means adding one row rather than changing the scoring logic. The extract below is the core of `/Shared/orders/01_quality_checks`.

```

# /Shared/orders/01_quality_checks
import json
from pyspark.sql import functions as F, types as T

dbutils.widgets.text('windowDate', '')
dbutils.widgets.text('rawPath', '')
window_date = dbutils.widgets.get('windowDate')
raw_path = dbutils.widgets.get('rawPath')

EXPECTED_COLUMNS = ['order_id', 'customer_id', 'order_date', 'status',
                    'ship_to_country', 'ship_to_postcode', 'carrier',
                    'service_level', 'quantity', 'unit_price', 'currency',
                    'weight_kg', 'source_system', 'extracted_at']
VALID_STATUS = ['CREATED', 'PICKED', 'SHIPPED', 'DELIVERED', 'CANCELLED']

df = (spark.read.option('header', True).option('mode', 'PERMISSIVE')
     .csv(raw_path).cache())
total = df.count()

if total == 0:
    dbutils.notebook.exit(json.dumps({'verdict': 'REJECT', 'qualityScore': '0.0000',
                                     'rowsRead': '0', 'failedRules': 'EMPTY_FILE'}))

schema_ok = df.columns == EXPECTED_COLUMNS # DQ-01 is file-level: right or not
countries = [r.code for r in spark.table('reference.iso_country').collect()]
dup_ids = [r.order_id for r in
           df.groupBy('order_id').count().filter('count > 1').collect()]

# Each row-level rule is a boolean column: True means the row passes.
RULES = [
    ('DQ-02', 0.20, F.col('order_id').isNotNull() & ~F.col('order_id').isin(dup_ids)),
    ('DQ-03', 0.20, F.col('order_id').isNotNull() & F.col('customer_id').isNotNull() &
     F.col('order_date').isNotNull() & F.col('ship_to_country').isNotNull() &
     F.col('quantity').isNotNull() & F.col('unit_price').isNotNull()),
    ('DQ-04', 0.10, F.to_date('order_date', 'yyyy-MM-dd').between(
     F.date_sub(F.lit(window_date), 30), F.lit(window_date))),
    ('DQ-05', 0.10, F.col('quantity').cast('int') > 0),
    ('DQ-06', 0.05, F.col('unit_price').cast('double').between(0, 100000)),
    ('DQ-07', 0.05, F.col('ship_to_country').isin(countries)),
    ('DQ-08', 0.05, F.col('status').isin(VALID_STATUS)),
]

results, score = [], 0.25 if schema_ok else 0.0
passed = df.agg(*[F.sum(F.when(expr, 1).otherwise(0)).alias(rid)
                  for rid, _, expr in RULES]).collect()[0].asDict()

for rule_id, weight, _ in RULES:
    pass_rate = passed[rule_id] / total
    score += weight * pass_rate
    results.append({'ruleId': rule_id, 'weight': weight,
                  'rowsPassed': passed[rule_id], 'rowsRead': total,
                  'passRate': round(pass_rate, 4),
                  'contribution': round(weight * pass_rate, 4)})

results.insert(0, {'ruleId': 'DQ-01', 'weight': 0.25,
                  'passRate': 1.0 if schema_ok else 0.0,
                  'contribution': 0.25 if schema_ok else 0.0})

report_path = f'{CURATED}/quality/orders/window_date={window_date}'
(spark.createDataFrame(results)
 .coalesce(1).write.mode('overwrite').json(report_path))

failed = ','.join(r['ruleId'] for r in results if r['passRate'] < 1.0)
dbutils.notebook.exit(json.dumps({
    'verdict': 'PASS' if score >= 0.80 else 'REJECT',
    'qualityScore': f'{score:.4f}',
    'rowsRead': str(total),
    'failedRules': failed,
    'reportPath': report_path,
}))

```

The notebook returns `verdict` for readability in alerts, but the pipeline does not act on it — the gate re-evaluates the score against the pipeline's own `qualityThreshold` parameter so that an authorised override works without editing the notebook.

5. Spark Transformation

Transformation runs only on data that has passed the quality gate. Its job is to turn the partner's flat extract into the conformed, typed, deduplicated records that reporting expects, and to publish them to `curated.fact_orders` in a way that a rerun cannot duplicate.

5.1 The output contract

Downstream reports depend on this table's shape. Any change to it is a breaking change and follows the change control process in Section 6.7.

Column	Type	Derived from	Notes
order_id	string	order_id, trimmed and upper-cased	Merge key. Unique across the whole table, not just within a day.
order_date	date	order_date parsed as yyyy-MM-dd	Partition column.
customer_id	string	customer_id, trimmed	
customer_segment	string	Looked up from <code>reference.dim_customer</code>	UNKNOWN when the customer is not yet in the dimension — a lookup miss never fails the run.
status	string	status, upper-cased	Constrained to the five values in DQ-08.
ship_to_country	string	ship_to_country, upper-cased	ISO 3166-1 alpha-2.
carrier	string	carrier, trimmed	
quantity	int	quantity cast to integer	
unit_price	decimal(12,2)	unit_price cast	Stored in the order's original currency.
net_amount	decimal(14,2)	quantity * unit_price	Computed here so every report agrees on the figure.
currency	string	currency, upper-cased	
weight_kg	decimal(10,3)	weight_kg cast	Null is permitted; it is not a mandatory field.
source_system	string	source_system	
window_date	date	Pipeline parameter <code>windowDate</code>	The business date the record was delivered for. Makes it possible to see which run produced a row.
ingested_at	timestamp	Notebook start time in UTC	Audit column.

5.2 Notebook activity configuration

```
{
  "name": "Transform_Orders",
  "type": "DatabricksNotebook",
  "linkedServiceName": {
    "referenceName": "ls_databricks_jobs",
    "type": "LinkedServiceReference"
  },
  "policy": {
    "timeout": "02:00:00",
    "retry": 1,
    "retryIntervalInSeconds": 300,
    "secureInput": false,
    "secureOutput": false
  },
  "typeProperties": {
    "notebookPath": "/Shared/orders/02_transform_orders",
    "baseParameters": {
      "windowDate": "@pipeline().parameters.windowDate",
      "rawPath": "@{concat(pipeline().globalParameters.rawRoot,
        'orders/ingest_date=',
        pipeline().parameters.windowDate, '/')}",
      "curatedPath": "@{concat(pipeline().globalParameters.curatedRoot,
        'orders/fact_orders')}",
      "runMode": "@pipeline().parameters.runMode",
      "runId": "@pipeline().RunId"
    },
    "libraries": []
  }
}
```

`rawRoot` and `curatedRoot` are data factory global parameters holding the `abfss://` prefixes for the environment — `abfss://raw@adlsmeridianprod.dfs.core.windows.net/` and `abfss://curated@adlsmeridianprod.dfs.core.windows.net/` in PROD. Keeping them out of the activity is what allows the same pipeline definition to be released unchanged to DEV, UAT and PROD.

Setting	Value	Why
<code>notebookPath</code>	<code>/Shared/orders/02_transform_orders</code>	An absolute workspace path. It must start with <code>/</code> ; a relative path is the most common cause of a <code>RESOURCE_DOES_NOT_EXIST</code> failure after a workspace reorganisation.
<code>baseParameters</code>	Five values, all passed from pipeline parameters	Nothing about the run is hard-coded in the notebook, so the same notebook serves scheduled runs, manual recoveries and backfills.
<code>timeout</code>	<code>02:00:00</code>	Roughly six times the observed run time, which leaves room for a slow day without letting a genuinely hung job run until morning.
<code>retry / interval</code>	<code>1 / 300</code>	One retry after five minutes covers a failed cluster acquisition. The merge is idempotent, so the retry cannot corrupt the table.
<code>libraries</code>	empty	The notebook uses only the runtime's built-in libraries. Keeping this empty removes library installation as a startup failure mode.

5.3 Cluster configuration

Both notebooks run on a job cluster created for the run and discarded afterwards. No interactive cluster is involved, so a user cannot affect the nightly run by restarting or reconfiguring a shared cluster.

Setting	Value	Reason
Cluster type	New job cluster per activity	Isolation and cost. Nothing runs between pipeline executions.
Databricks runtime	14.3 LTS (Spark 3.5, Scala 2.12)	Long-term support release; upgrades are a planned change, never automatic.
Driver / worker node	Standard_D8ds_v5	Sized for a file of roughly 120,000 rows with headroom for growth.
Workers	Autoscale 2 to 6	The lower bound keeps a small file fast; the upper bound caps the cost of an unusually large one.

Setting	Value	Reason
Cluster policy	policy-jobs-standard	Enforces tagging, the maximum node count and the auto-termination rule.
Tags	costCentre=logistics-analytics, pipeline=pl_ProcessDailyOrders	Makes the pipeline's compute spend attributable in cost reporting.
Access mode	Single user, running as the factory's managed identity	The notebook reaches storage through the identity's role assignment; no credential is stored in the notebook.

5.4 Parameters inside the notebook

Values in `baseParameters` arrive as Databricks widgets. Declaring every widget with an empty default and reading it explicitly means a missing parameter fails immediately and visibly, rather than silently processing the wrong day.

```
dbutils.widgets.text('windowDate', '')
dbutils.widgets.text('rawPath', '')
dbutils.widgets.text('curatedPath', '')
dbutils.widgets.text('runMode', 'scheduled')
dbutils.widgets.text('runId', '')

window_date = dbutils.widgets.get('windowDate')
raw_path = dbutils.widgets.get('rawPath')
curated_path = dbutils.widgets.get('curatedPath')

assert window_date, 'windowDate was not supplied by the pipeline'
```

5.5 What the transformation does

1. **Read** the raw copy for the business date with an explicit schema. An explicit schema means a partner adding a column changes nothing here — DQ-01 has already rejected that file before this notebook runs.
2. **Standardise** text fields: trim whitespace, upper-case the codes (`status`, `ship_to_country`, `currency`), and normalise empty strings to null so that "missing" has one representation.
3. **Type** the numeric and date columns: `quantity` to integer, `unit_price` and `weight_kg` to decimal, `order_date` to date.
4. **Deduplicate** within the file, keeping the record with the latest `extracted_at` for each `order_id`. A same-day correction from the partner arrives as a second row rather than a separate file.
5. **Enrich** with `customer_segment` from `reference.dim_customer` using a left join, so an unknown customer yields `UNKNOWN` rather than dropping the order.
6. **Derive** `net_amount` as `quantity * unit_price`, computed once here so every downstream report agrees.
7. **Merge** into `curated.fact_orders` on `order_id`, updating existing rows and inserting new ones.
8. **Return** the row counts to the pipeline so they can be written to the audit table.

5.6 The transformation notebook

```
# /Shared/orders/02_transform_orders
import json
from datetime import datetime, timezone
from delta.tables import DeltaTable
from pyspark.sql import functions as F, types as T, Window

# All 14 source columns are read as strings and typed explicitly below,
# so a malformed value becomes null rather than failing the whole read.
SCHEMA = T.StructType([T.StructField(c, T.StringType()) for c in EXPECTED_COLUMNS])

raw = (spark.read.option('header', True).schema(SCHEMA).csv(raw_path))
rows_read = raw.count()

def blank_to_null(col):
    return F.when(F.trim(F.col(col)) == '', None).otherwise(F.trim(F.col(col)))

clean = (raw
    .withColumn('order_id', F.upper(blank_to_null('order_id')))
    .withColumn('customer_id', blank_to_null('customer_id'))
    .withColumn('status', F.upper(blank_to_null('status')))
    .withColumn('ship_to_country', F.upper(blank_to_null('ship_to_country')))
    .withColumn('currency', F.upper(blank_to_null('currency')))
    .withColumn('carrier', blank_to_null('carrier'))
    .withColumn('order_date', F.to_date('order_date', 'yyyy-MM-dd'))
    .withColumn('quantity', F.col('quantity').cast(T.IntegerType()))
    .withColumn('unit_price', F.col('unit_price').cast(T.DecimalType(12, 2)))
    .withColumn('weight_kg', F.col('weight_kg').cast(T.DecimalType(10, 3)))
    .withColumn('extracted_at', F.to_timestamp('extracted_at')))

# Keep the latest correction per order within this file.
latest = Window.partitionBy('order_id').orderBy(F.col('extracted_at').desc())
deduped = (clean.withColumn('_rn', F.row_number().over(latest))
    .filter(F.col('_rn') == 1).drop('_rn'))

customers = spark.table('reference.dim_customer').select('customer_id', 'customer_segment')

final = (deduped.join(F.broadcast(customers), 'customer_id', 'left')
    .withColumn('customer_segment', F.coalesce('customer_segment', F.lit('UNKNOWN')))
    .withColumn('net_amount', (F.col('quantity') * F.col('unit_price')).cast(T.DecimalType(14, 2)))
    .withColumn('window_date', F.lit(window_date).cast(T.DateType()))
    .withColumn('ingested_at', F.lit(datetime.now(timezone.utc)))
    .select('order_id', 'order_date', 'customer_id', 'customer_segment', 'status',
        'ship_to_country', 'carrier', 'quantity', 'unit_price', 'net_amount',
        'currency', 'weight_kg', 'source_system', 'window_date', 'ingested_at'))

rows_written = final.count()

if DeltaTable.isDeltaTable(spark, curated_path):
    (DeltaTable.forPath(spark, curated_path).alias('t')
        .merge(final.alias('s'), 't.order_id = s.order_id')
        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute())
else:
    (final.write.format('delta').mode('overwrite')
        .partitionBy('order_date').save(curated_path))

spark.sql(f"OPTIMIZE delta.`{curated_path}` "
    f"WHERE order_date >= date_sub('{window_date}', 7)")

dbutils.notebook.exit(json.dumps({
    'rowsRead': str(rows_read),
    'rowsWritten': str(rows_written),
    'rowsDeduped': str(rows_read - rows_written),
    'windowDate': window_date,
}))
```

5.7 Reading the result back in the pipeline

The values returned above are read by `Log_Run_Success` and written to the audit table, using the same `runOutput` mechanism the quality gate uses:

```
@{activity('Transform_Orders').output.runOutput.rowsWritten}
@{activity('Transform_Orders').output.runOutput.rowsDeduped}
```

`rowsDeduped` is worth watching. A sudden rise means the partner is sending many same-day corrections, which is usually the first visible symptom of a problem in their upstream system.

5.8 Performance and layout

- The table is partitioned by `order_date`. Reports filter on it almost every time, and a day's partition is a natural unit for reprocessing.
- `window_date` is stored as a column rather than a second partition. Partitioning by both would create small files, because most of a day's records share one `order_date`.
- The customer dimension is broadcast. It is a few thousand rows, so broadcasting avoids a shuffle on every run.
- `OPTIMIZE` is scoped to the last seven days rather than the whole table. Compaction is only useful where recent merges have created small files, and an unscoped `OPTIMIZE` would grow into the longest step of the run.
- A typical run reads about 120,000 rows and completes in four to six minutes on the cluster in Section 5.3, of which two to three minutes is cluster startup.

5.9 Schema changes

The notebook reads with an explicit schema and does not enable automatic schema merging. A partner adding a column is therefore stopped one step earlier, by DQ-01, and never reaches this notebook. Adopting a new column is a deliberate change with a fixed order: agree the change with the partner, update `EXPECTED_COLUMNS` in the quality notebook and `SCHEMA` here, extend the curated table, update Section 5.1 of this document, and release all of it together. The pipeline is designed so that skipping any of those steps rejects files rather than corrupting the table.

6. Operational Runbook

This section is written to be used during an incident by someone who did not build the pipeline. It assumes only the access listed at the front of this document and no knowledge of the pipeline's internals beyond what is written here.

6.1 Daily health check

Five minutes each working morning, before 09:00 local time. The purpose is to catch the failures that did not alert — chiefly a run that never started.

#	Check	What a healthy result looks like
1	In the Data Factory Monitor → Pipeline runs tab, filter to the last 24 hours.	Exactly one run of <code>pl_Orchestrate_DailyOrders</code> and one of <code>pl_ProcessDailyOrders</code> , both Succeeded , started at 02:00 UTC.
2	Open the successful run and read the <code>Run_Quality_Checks</code> output.	<code>qualityScore</code> at or above 0.80. Note the value — a score that is drifting downward week on week is worth raising with the partner before it crosses the threshold.
3	Read the <code>Transform_Orders</code> output.	<code>rowsWritten</code> within roughly 20% of the same weekday last week. A sharp drop usually means a truncated file rather than a quiet day.
4	Confirm the trigger is still running.	<code>runtimeState</code> is <code>Started</code> (command in Section 2.5). A trigger left stopped after maintenance is the single most common cause of a missed day.
5	Scan the operations mailbox for Warning alerts from the last 24 hours.	None. A Warning means a file was rejected and needs following up even though nothing failed.

6.2 Where to look

Surface	How to reach it	What it tells you
Data Factory Monitor	<code>adf-meridian-prod</code> → Monitor → Pipeline runs. Search by the <code>childRunId</code> from the alert email.	Which activity failed, the error message, the duration of each activity, and how many retry attempts were made
Activity input / output	In the run detail, the input and output icons on each activity	The exact paths and parameters used, and the values a notebook returned. This is the fastest way to confirm the run used the business date you expected.
Databricks run page	The <code>Run</code> page URL in the notebook activity's output	Spark logs, the notebook's own output, and the cluster event log — the only place a cluster startup failure is fully explained
Quality report	<code>curated/quality/orders/window_date=<date>/</code>	Per-rule pass rates and example failing order IDs for a rejected file
Quarantine	<code>landing/quarantine/<windowDate>/</code>	The rejected file itself, ready to send back to the partner
Log Analytics	<code>log-meridian-prod</code> → Logs. Queries in Appendix A.	History across runs: how often an activity retries, whether durations are trending, whether a run happened at all
Audit table	<code>sqldb-meridian-meta</code> , <code>audit.pipeline_run</code>	One row per run with outcome, score, threshold applied and row counts — the record used when someone asks what happened three weeks ago

6.3 Reading a failed run

1. Open the run from the `monitorUrl` in the alert email, or search the Monitor tab for the `childRunId`.
2. Find the first activity with a red **Failed** icon. Later activities showing **Skipped** are a consequence, not a second problem.

3. Hover the failed activity and open the error detail. Note the `ErrorCode` — it is what selects the scenario below.
4. Check the number of attempts shown. If the activity retried and still failed, the problem is not transient and waiting will not fix it.
5. Confirm `windowDate` in the activity's input matches the business date you expect to be recovering.
6. Match the error to a scenario in Section 6.4 and follow it. If nothing matches, escalate under Section 6.6 rather than improvising — the pipeline's safety properties depend on it being rerun the documented way.

6.4 Failure scenarios

ID	Scenario	Alert received	Most common cause
S-01	The order file did not arrive	Critical	The partner's transfer job failed or ran late
S-02	The file was rejected by the quality gate	Warning	A schema change at the partner, or a bad extract
S-03	The Databricks notebook failed	Critical	Cluster could not start, or a reference table was unavailable
S-04	Storage or authentication failure	Critical	An expired secret or a changed role assignment
S-05	No run happened at all	Critical (missed-run alert)	The trigger was left stopped after maintenance

S-01 The order file did not arrive

Symptom	<code>Wait_For_Order_File</code> fails after 90 minutes with a validation timeout. Every later activity is skipped.
Alert	Critical, at approximately 03:30 UTC — 90 minutes after the trigger fires.
Impact	<code>curated.fact_orders</code> has no data for the business date. Reports for that day will be short until the run completes.

Diagnose

1. List the landing folder for the business date: `landing/orders/2026/08/`. Confirm whether the file is absent, or present under an unexpected name.
2. If the file is present but named differently — a `_v2` suffix, a different date format, an unexpected case — this is a naming problem, not an availability problem. Go to step 3 of the resolution.
3. If the file is absent, check the partner transfer job's own status page, then contact the partner operations desk (Section 6.6).

Resolve

1. If the partner confirms the file is delayed and will arrive within a few hours, wait for it, then run the recovery in Section 6.5 with `windowDate` set to the business date and `runMode` set to `manual`.
2. If the file will not arrive at all that day, record it in the incident ticket and tell the reporting team that the business date will be missing. Do not lower the quality threshold to force a partial file through.
3. If the file is present under the wrong name, do **not** rename the partner's file in place — that breaks their reconciliation. Copy it to the expected name in the same folder, then run the recovery.

Prevention. A file that arrives late two days in a row is a pattern, not a coincidence. Raise it with the partner rather than absorbing it with manual recoveries; if their schedule has genuinely moved, the trigger time should move with it under Section 6.7.

S-02 The file was rejected by the quality gate

Symptom	A Warning email naming the score and the failed rules. The pipeline run shows Succeeded ; there is no failed activity to find.
Alert	Warning, roughly 10 minutes after the trigger fires.
Impact	No data published for the business date. The previous day remains the latest data in the curated table.

Diagnose

1. Open the quality report at [curated/quality/orders/window_date=<date>/](#). It lists every rule with its pass rate and contribution.
2. Identify which rule cost the most. The [contribution](#) column makes this immediate — the rule furthest below its weight is the one to explain.
3. If **DQ-01** failed, the header does not match the agreed contract. Compare the file's header with [EXPECTED_COLUMNS](#) in Section 4.8. A schema change alone caps the score at 0.75, so nothing else needs investigating.
4. If row-level rules failed, take the example [order_id](#) values from the report — these are what make the conversation with the partner concrete.

Resolve

1. Send the partner the quality report summary and the example order IDs, and ask for a corrected extract for the business date.
2. When the corrected file arrives, confirm it is in landing under the expected name, then run the recovery in Section 6.5.
3. If DQ-01 failed because of an intentional partner schema change, this is a change request, not an incident. Raise it with Data Engineering under Section 6.7 — the notebooks and the curated table must be updated together before any file with the new schema can be accepted.
4. If the partner insists the data is correct and the rules are wrong, that is a governance question. Follow Section 4.7; do not override the threshold on your own authority.

S-03 The Databricks notebook failed

Symptom	Run_Quality_Checks or Transform_Orders fails. The activity output contains a Run page URL .
Alert	Critical.
Impact	No data published. The raw copy is already safe, so nothing needs to be re-fetched from the partner.

Diagnose

1. Open the [Run page URL](#) from the activity output. It leads to the Databricks run, which carries the real error; the Data Factory message is only a summary.
2. Read the cluster event log first. [INIT_SCRIPT_FAILURE](#), [CLOUD_PROVIDER_LAUNCH_FAILURE](#) or a quota message means the notebook never ran — this is a compute problem, not a data problem.

3. If the cluster started, read the notebook error. `AnalysisException` on `reference.dim_customer` or `reference.iso_country` means a reference table was unavailable or was being rewritten at the time.
4. `RESOURCE_DOES_NOT_EXIST` means the notebook path in the activity no longer resolves — usually a workspace folder that was moved or renamed.

Resolve

1. For a cluster launch failure: check the subscription's core quota in the region, then rerun using Section 6.5. These failures are frequently transient and clear on the next attempt.
2. For a reference table failure: confirm with the team that owns `reference` that the table is available, then rerun.
3. For `RESOURCE_DOES_NOT_EXIST`: do not move the notebook back yourself. Escalate to Data Engineering — the path is part of the released pipeline definition and must be corrected through deployment.
4. For any error inside the notebook's own logic, escalate to Data Engineering with the Databricks run URL. Do not edit the notebook to get past it; a workspace edit is overwritten by the next release and leaves no record.

S-04 Storage or authentication failure

Symptom	<code>Copy_Landing_To_Raw</code> Or <code>Quarantine_Rejected_File</code> fails after exhausting its retries, with <code>AdlsGen2OperationFailed</code> , <code>403 Forbidden</code> , or <code>UserErrorFailedToConnectToKeyVault</code> .
Alert	Critical.
Impact	Depends on where it failed. A failure of the raw copy means nothing was processed; a failure of the quarantine copy means a rejected file was not archived, though the Warning alert was still sent.

Diagnose

1. Distinguish availability from permission. An HTTP 503 or a timeout is availability and the retries have already tried; a 403 is permission and no number of retries will help.
2. For a 403 on storage, confirm the factory's managed identity still holds **Storage Blob Data Contributor** on `adlsmeridianprod`. Role assignments are sometimes removed by subscription-wide access reviews.
3. For a Key Vault error, check whether the secret has expired and whether the factory's identity still has Get and List permissions on `kv-meridian-prod`.
4. Check the Azure status page for the storage region before assuming the problem is local to this pipeline.

Resolve

1. Availability: wait for the platform issue to clear, then rerun with Section 6.5. Nothing needs to be cleaned up first — the raw path is overwritten by the rerun.
2. Permission: raise a Sev 2 with the Platform team to restore the role assignment or renew the secret. Rerun once they confirm. This cannot be fixed by the operations team alone.
3. If only the quarantine copy failed, the rejected file is still in landing and the partner has already been notified. Rerunning is optional; recovering the archive copy matters less than following up the data problem.

S-05 No run happened at all

Symptom	The missed-run alert fires at 03:15 UTC, or the daily health check finds no run for the previous night. There is no failed run to open, because no run was ever created.
Alert	Critical, from the Azure Monitor log alert rather than from the pipeline.
Impact	No data published, and — unlike every other scenario — nothing in the pipeline's own history to explain why.

Diagnose

1. Check the trigger's runtime state with the command in Section 2.5. **Stopped** is by far the most common cause, usually a maintenance window that was never closed out.
2. If the trigger is **Started**, open **Monitor → Trigger runs** and look for a failed trigger run. A trigger can fail to start a pipeline if the pipeline reference is invalid after a bad release.
3. If neither, check whether the data factory itself was unavailable during the window on the Azure status page.

Resolve

1. Start the trigger if it is stopped, and confirm the state afterwards. A schedule trigger does not catch up, so starting it does **not** recover the missed window.
2. Recover the missed window with Section 6.5, using the business date that was missed.
3. If a release broke the trigger's pipeline reference, escalate to Data Engineering; the fix is a redeployment, not a portal edit.
4. Record in the incident ticket why the trigger was stopped and who stopped it. Repeated occurrences of this scenario are a process problem in change management, not a pipeline problem.

6.5 Rerunning and backfilling

Rerunning a single failed run

Data Factory offers **Rerun from failed activity**, which resumes the existing run rather than starting a new one. It is the right choice when the cause was transient and nothing about the inputs has changed.

1. Open the failed run in the Monitor tab.
2. Choose **Rerun from failed activity**. Activities that already succeeded are not repeated.
3. Watch the run to completion. The parameters, including **windowDate**, are inherited from the original run, so there is nothing to re-enter and nothing to get wrong.

When not to use rerun-from-failed. If the input file has been replaced — a corrected extract from the partner, or a file that was renamed — start a fresh manual run instead. Rerun-from-failed resumes mid-pipeline and may skip the copy that would have picked up the new file.

Recovering a window with a fresh run

1. Confirm the corrected or delayed file is in landing under the expected name for the business date.
2. Trigger **pl_Orchestrate_DailyOrders** manually (Section 2.6).
3. Set **windowDate** to the **business date**, **runMode** to **manual**, **scheduledRunTime** to the timestamp of the window being replaced, and leave **qualityThreshold** at **0.80**.
4. Watch the run. On success, confirm **rowsWritten** in the **Transform_Orders** output is plausible for that day.
5. Confirm the audit row for the business date now shows the successful outcome, and close the incident ticket with the run ID.

Backfilling several days

1. Confirm every file for the range is present in landing. Missing days must be resolved individually first — a backfill will not wait for a file.

- Run the dates **one at a time, oldest first**, with `runMode` set to `backfill`. Concurrency is limited to 1 for a reason: two runs writing overlapping `order_date` partitions is the one situation the design does not make safe.
- Check each run's outcome before starting the next. A file rejected mid-range must be resolved before continuing, or the range will have a hole in it.
- When the range is complete, ask the reporting team to confirm their totals reconcile for the whole period, not just the last day.

6.6 Escalation

Escalate when the resolution steps in a scenario do not apply, when they have been followed and the run still fails, or immediately for anything marked below as requiring another team.

Situation	Severity	Contact	Response expectation
Pipeline failure the runbook covers	Sev 3	Operations team, self-service	Resolved during the shift
Repeated failure after a documented rerun	Sev 2	Data Engineering on-call	30 minutes during business hours; 1 hour overnight
Storage, Key Vault or role assignment failure	Sev 2	Platform / Cloud team	30 minutes
Missing or late file, or a quality rejection needing a resend	Sev 3	Partner operations desk, copying the Operations Manager	Same business day
Partner schema change	Sev 3 (change, not incident)	Data Engineering, via change request	Assessed within 2 business days
Threshold or rule change request	n/a	Data Governance forum	Next scheduled forum
Two or more consecutive days without published data	Sev 1	Data Engineering Lead and Operations Manager together	Immediate

6.7 Changes and maintenance

The pipeline is deployed from source control. Nothing in it is edited in the portal, including during an incident: a portal edit is silently reverted by the next release, and the difference between the two is invisible until something fails.

Change	Route	Notes
Trigger time	Change request to Data Engineering	Stop the trigger, publish, start it again, and confirm the runtime state. Update Section 2.1 of this document in the same change.
Quality rule or weight	Data Governance forum, then Data Engineering	Weights must still sum to 1.00. Section 4.2 is updated in the same release.
Quality threshold	Data Governance forum	See Section 4.7 for the one-off override route.
Curated table schema	Change request; requires downstream impact assessment	Breaking change for reporting. The order of operations is in Section 5.9.
Planned maintenance window	Notify the operations mailbox in advance	Stop the trigger before, start it after, and confirm the state. Then recover any missed window under Section 6.5.
Databricks runtime upgrade	Change request	Tested in UAT against a full week of files before it reaches PROD.

6.8 Things not to do

- Do not delete or rename files in the landing container.** It is owned by the partner transfer process and is used for their reconciliation. Copy instead.

- **Do not edit the notebooks in the Databricks workspace to get a run through.** The change is lost at the next release and leaves no record of what published the data.
- **Do not lower `qualityThreshold` to force a rejected file through.** The threshold exists to stop exactly that decision being made at 03:00 by one person. Follow Section 4.7.
- **Do not run two business dates at the same time.** Concurrency is limited to 1, but a second manual run can still be queued; run backfills sequentially.
- **Do not delete files from quarantine.** They are the evidence for the conversation with the partner and are cleared automatically after 90 days.
- **Do not leave a trigger stopped without a reminder to start it.** Scenario S-05 exists because this is the most common way a day of data is lost.

Appendix A — Diagnostic queries

These run against `log-meridian-prod` in Log Analytics. Diagnostic settings on `adf-meridian-prod` send `PipelineRuns`, `ActivityRuns` and `TriggerRuns` to this workspace with a 90-day retention.

Did the run happen, and how did it end?

```
ADFPipelineRun
| where TimeGenerated > ago(7d)
| where PipelineName == "pl_ProcessDailyOrders"
| project TimeGenerated, RunId, Status, Start, End,
    DurationMs = datetime_diff('second', End, Start),
    WindowDate = tostring(parse_json(Parameters).windowDate)
| order by TimeGenerated desc
```

Which activities failed, and with what error?

```
ADFActivityRun
| where TimeGenerated > ago(7d)
| where Status == "Failed"
| project TimeGenerated, PipelineName, ActivityName, ActivityType,
    ErrorCode = tostring(parse_json(Error).errorCode),
    ErrorMessage = tostring(parse_json(Error).message)
| order by TimeGenerated desc
```

How is the quality score trending?

```
ADFActivityRun
| where TimeGenerated > ago(60d)
| where ActivityName == "Run_Quality_Checks" and Status == "Succeeded"
| extend Score = todouble(parse_json(tostring(parse_json(Output).runOutput)).qualityScore)
| project TimeGenerated, Score
| render timechart
```


Appendix B — Glossary

Term	Meaning
Backfill	Reprocessing a range of past business dates deliberately, as opposed to recovering a single failed run.
Business date	The date the orders belong to. The run at 02:00 UTC on 21 August processes the business date 20 August.
Curated zone	The published, conformed data that reporting reads. Written only by the transformation notebook.
Dependency condition	The rule that decides whether the next activity runs: <i>Succeeded</i> , <i>Failed</i> , <i>Completed</i> or <i>Skipped</i> .
Idempotent	Safe to run more than once: the second run produces the same result as the first rather than duplicating its effect.
MERGE	An update-or-insert operation. It is what makes a rerun of the same business date replace records instead of duplicating them.
Quality score	A weighted number between 0 and 1 summarising how well a file met the rule set. Compared against the threshold of 0.80.
Quarantine	Where a rejected file is archived, together with the report explaining why it was rejected.
Raw zone	The immutable dated copy of every file received. The source for any reprocessing.
runOutput	The JSON a Databricks notebook returns to the pipeline through <code>dbutils.notebook.exit</code> . Capped at 2 MB.
Trigger	The schedule that starts the pipeline. A schedule trigger fires forward only and never catches up on missed windows.

Appendix C — References

- Microsoft Learn, **Pipelines and activities in Azure Data Factory** — activity policy properties (`timeout`, `retry`, `retryIntervalInSeconds`, `secureOutput`) and the four dependency conditions.
- Microsoft Learn, **Create a trigger that runs a pipeline on a schedule** — the `ScheduleTrigger` JSON schema, `recurrence` fields, and passing trigger metadata to pipeline parameters.
- Microsoft Learn, **Pass trigger information to pipeline** — `@trigger().scheduledTime` and related system variables.
- Microsoft Learn, **Transform data by running a Databricks notebook** — the `DatabricksNotebook` activity, `baseParameters`, and reading `dbutils.notebook.exit` output through `runOutput`.
- Microsoft Learn, **Monitor Data Factory** and **Data Factory monitoring data reference** — the `PipelineFailedRuns` metric and the `ADFPipelineRun` / `ADFActivityRun` Log Analytics tables.

End of document. Corrections and gaps found during real incidents should be raised with Data Engineering so that the next reader is better served than this one was.